

Module 2: Variables, Data Types & Operators

TARGET: CAMPUS & OFF-CAMPUS PLACEMENTS

DIFFICULTY: FOUNDATIONAL TO ADVANCED TRAPS

WEIGHTAGE: 100% IN MCQS & CODING SCREENINGS

1. Placement Learning Objectives & Readiness Checklist

By the end of this module, a B.Tech candidate must master not just the syntax, but the **underlying JVM memory model, evaluation rules, and interview traps**:

- ✓ Explain variable lifecycle and memory allocation (Stack vs Heap).

✓ Differentiate between the 8 Primitives and Reference Types.

✓ Predict exact type promotion in binary and unary expressions.

✓ Explain Definite Assignment Analysis for local variables.

✓ Master Wrapper classes, Autoboxing, Unboxing, and NPE pitfalls.

✓ Explain the Integer Cache mechanism (`-128` to `127`).

✓ Distinguish `==` (reference check) from `.equals()` (value check).
- ✓ Master Bitwise operator tricks (XOR, Bit masking, Two's complement).

✓ Understand short-circuiting (`&&` , `||` vs `&` , `|`).

✓ Explain shift operators and the 5-bit shift distance mask (`1 << 32`).

✓ Distinguish prefix vs postfix (`++x` vs `x++`) evaluation order.

✓ Understand Implicit Narrowing in compound assignment (`b += 1`).

✓ Explain floating-point IEEE-754 precision loss and `BigDecimal` .

✓ Solve classic bitwise and mathematical placement coding questions.

2. Variables in Java: The Mental & Memory Model

A **variable** is an identifier associated with a specific memory location allocated to store values of a declared type. Java is a **statically typed language**, meaning variable types are strictly bound and verified at compile time.

```
int age = 25;
// age = "25"; ← COMPILE-TIME ERROR: Type mismatch (cannot convert from String to int)
```

Variable Declaration Anatomy:

int	age	=	25;
(Data Type)	(Variable Name)	(Assignment Op)	(Literal)

Memory (Stack Frame):

[age] → | 00000000 00000000 00000000 00011001 | (32 bits = 25)

3. Variable Types & Lifecycle Scopes

In Java, variables are categorized based on where they are declared and their lifecycle in JVM memory:

Variable Type	Where Declared	Memory Area	Default Value?	Lifecycle
Local Variable	Inside method, constructor, or block <code>{ ... }</code>	Stack Frame	NO (Must be explicitly initialized)	Created on method call, destroyed on frame exit.
Instance Variable	Inside class, outside any method (non-static)	Heap (part of Object state)	YES (0, 0.0, false, null)	Created with <code>new</code> , destroyed by Garbage Collector.
Static / Class Variable	Inside class, declared with <code>static</code> keyword	Metaspace (Class Area)	YES (0, 0.0, false, null)	Loaded when Class is loaded; unloaded with ClassLoader.
Parameter	Method / constructor signature	Stack Frame	Initialized by caller arguments	Scope limited to method body.

INTERVIEW TRAP: Definite Assignment Analysis

Unlike C/C++ where uninitialized local variables hold "garbage values", Java local variables **produce a compile-time error** if read before initialization. The Java compiler verifies through control flow analysis whether every possible execution path initializes the variable.

```
public void checkAssignment(boolean condition) {
    int x;
    if (condition) {
        x = 10;
    }
    // System.out.println(x); // COMPILE ERROR: The local variable x may not have been initialized
}
```

4. The 8 Primitive Data Types (Exhaustive Reference)

Java has exactly **8 primitive types**. Primitives store raw binary values directly in memory (no object headers or pointer indirection).

Type	Bits	Bytes	Minimum Value	Maximum Value	Default Value	Wrapper Class
<code>byte</code>	8	1	$-2^7 = -128$	$2^7 - 1 = 127$	<code>0</code>	<code>Byte</code>
<code>short</code>	16	2	$-2^{15} = -32,768$	$2^{15} - 1 = 32,767$	<code>0</code>	<code>Short</code>
<code>int</code>	32	4	$-2^{31} \approx -2.14 \times 10^9$	$2^{31} - 1 \approx 2.14 \times 10^9$	<code>0</code>	<code>Integer</code>
<code>long</code>	64	8	$-2^{63} \approx -9.22 \times 10^{18}$	$2^{63} - 1 \approx 9.22 \times 10^{18}$	<code>0L</code>	<code>Long</code>
<code>float</code>	32	4	1.4×10^{-45} (IEEE 754)	3.4028235×10^{38}	<code>0.0f</code>	<code>Float</code>
<code>double</code>	64	8	4.9×10^{-324} (IEEE 754)	$1.79769313 \times 10^{308}$	<code>0.0d</code>	<code>Double</code>
<code>char</code>	16	2	<code>'\u0000'</code> (0)	<code>'\uffff'</code> (65,535)	<code>'\u0000'</code>	<code>Character</code>
<code>boolean</code>	*	*	<code>false</code>	<code>true</code>	<code>false</code>	<code>Boolean</code>

UNDER THE HOOD: Why boolean size is JVM-dependent

According to the *Java Language Specification (JLS §4.2.5)*, the size of a `boolean` is not specified. In Oracle/OpenJDK HotSpot:

- An individual `boolean` local variable on the stack is compiled into an `int` (using 4 bytes / 32 bits).
- In a `boolean[]` array, HotSpot packs elements as an array of `byte` (1 byte per boolean).
- **Interview Answer:** Never say "boolean is always 1 byte". State that the JLS does not fix the size; HotSpot typically uses 4 bytes for standalone variables and 1 byte in arrays.

5. Critical Interview Traps on Literals & Precision

Trap 1: The Floating-Point Literal Suffix Rule

```
float price = 10.5;    // ❌ COMPILE ERROR: Type mismatch (double cannot be converted to float)
float price = 10.5f;   // ✅ CORRECT: 'f' or 'F' suffix tells compiler it is a 32-bit float
double d      = 10.5;  // ✅ CORRECT: Floating-point decimal literals default to double
```

Trap 2: The Integer Literal Bounds & Suffix Rule

```
long x = 100;           // ✅ Valid: 100 is an int literal widened automatically to long
long pop = 8000000000;  // ❌ COMPILE ERROR: Integer number too large (evaluated as int literal first!)
long pop = 8000000000L; // ✅ CORRECT: 'L' suffix tells compiler to treat literal as a 64-bit long
```

Trap 3: IEEE-754 Precision Loss & Financial Calculations

```
double a = 0.1;
double b = 0.2;
System.out.println(a + b); // Prints: 0.30000000000000004 (NOT 0.3!)
```

🚨 PLACEMENT QUESTION: Why $0.1 + 0.2 \neq 0.3$ in Java?

Numbers like \$0.1\$ and \$0.2\$ cannot be represented precisely in base-2 binary floating point (they result in repeating binary fractions, just like $\$1/3$ in base-10). In monetary and FinTech applications, **always use** `java.math.BigDecimal` initialized with Strings:

```
BigDecimal d1 = new BigDecimal("0.1");
BigDecimal d2 = new BigDecimal("0.2");
System.out.println(d1.add(d2)); // Exactly: 0.3
```

Trap 4: `char` is an Unsigned 16-Bit Integer Under the Hood

```
char ch = 'A';
System.out.println(ch + 1);           // Prints: 66 ('A' promoted to ASCII/Unicode 65 + 1)
System.out.println((char)(ch + 1));   // Prints: 'B'

char c1 = 'A', c2 = 'B';
System.out.println(c1 + c2);           // Prints: 131 (65 + 66)
System.out.println("" + c1 + c2);     // Prints: "AB" (String concatenation triggers!)
```

6. Primitive vs Reference Types: Stack vs Heap Layout

Every non-primitive type in Java (including `String`, arrays `int[]`, and all classes) is a **Reference Type**.

Code:

```
int age = 25;
String name = new String("Rahul");
int[] arr = new int[]{10, 20};
```

STACK FRAME

age: [25]
name: [0x4A12]
arr: [0x8B34]

(raw value)

HEAP MEMORY

String Object @ 0x4A12 value: ['R','a','h','u','l']
int[] Array Object @ 0x8B34 length: 2, [10 20]

Comparison Criterion	Primitive Types	Reference Types
Data Stored	Direct raw binary representation	Memory address / pointer to object on Heap
Can Hold null ?	NEVER	YES (Unassigned reference points to null)
Default Values (fields)	0, 0.0, false, '\u0000'	null
Java Collections Compatibility	Cannot be used (<code>List<int></code> is invalid)	Directly supported (<code>List<Integer></code>)
== Comparison	Compares actual raw values	Compares memory addresses / object identity

7. Wrapper Classes, Autoboxing, Unboxing & NPE Trap

Java provides object-oriented wrappers for every primitive to enable interoperability with the Collections Framework, Generics, and Reflection:

Autoboxing: `int` —> `Integer.valueOf(x)` —> Integer Object

Unboxing: Integer Object —> `obj.intValue()` —> `int`

FATAL PLACEMENT TRAP: Unboxing a Null Wrapper Causes NullPointerException

```
Integer box = null;
int val = box; // Compiles cleanly! At runtime throws: java.lang.NullPointerException!
```

Why? The Java compiler rewrites `int val = box;` into `int val = box.intValue();`. Invoking any instance method on a `null` reference throws an immediate `NullPointerException`.

8. The Integer Cache Mechanism: Top Tier-1 Interview Question

Java optimizes memory allocation by caching `Integer` objects within the range `-128 to 127` (inclusive) in an internal static cache (`IntegerCache`) initialized during class loading.

```
Integer a = 100;
Integer b = 100;
System.out.println(a == b); // TRUE (Both point to identical cached object in IntegerCache)

Integer x = 128;
Integer y = 128;
System.out.println(x == y); // FALSE (128 is outside cache! Two distinct objects on Heap)
System.out.println(x.equals(y)); // TRUE (Value comparison!)
```

💡 GOLDEN PLACEMENT RULE FOR WRAPPER COMPARISON

NEVER use `==` to compare wrapper objects for mathematical equivalence!

- `==` compares object references (memory address).
- `.equals()` compares deep logical value.
- Alternatively, use `Objects.equals(a, b)` to safely guard against `NullPointerException`.

9. Operators in Java: Complete Breakdown

9.1 Arithmetic Operators & Division Hazards

Operation	Code	Result	Technical Explanation
Integer Division	<code>10 / 3</code>	<code>3</code>	Both operands are integers; fractional part is truncated towards zero.
Floating Division	<code>(double)10 / 3</code>	<code>3.333333333333335</code>	Type promotion forces 64-bit IEEE-754 floating division.
Integer Div by 0	<code>10 / 0</code>	ARITHMETICEXCEPTION	Throws runtime <code>java.lang.ArithmeticException: / by zero</code> .
Floating Div by 0	<code>10.0 / 0.0</code>	<code>Infinity</code>	IEEE-754 defines positive floating-point infinity.
Zero / Zero (Float)	<code>0.0 / 0.0</code>	<code>NaN</code>	Not-a-Number (undefined floating result).

9.2 Prefix vs Postfix Unary Operators

- `++x` (Prefix): Increment value **before** taking part in expression evaluation.
- `x++` (Postfix): Store current value for expression evaluation, increment **after**.

```
int x = 5;
int y = x++ + ++x;
// Evaluation Step-by-Step:
// 1. x++ evaluates to 5. Internal x becomes 6.
// 2. ++x increments internal x from 6 to 7, evaluates to 7.
// 3. y = 5 + 7 = 12. Final x = 7.
System.out.println("y = " + y + ", x = " + x); // y = 12, x = 7
```

9.3 Short-Circuit Evaluation (`&&`, `||` vs `&`, `|`)

Operator	Type	Short-Circuit Behavior
<code>&&</code>	Conditional AND	Short-circuits: If LHS is <code>false</code> , RHS is NEVER evaluated.
<code>&</code>	Bitwise / Boolean AND	No short-circuit: Always evaluates both LHS and RHS even if LHS is false.
<code> </code>	Conditional OR	Short-circuits: If LHS is <code>true</code> , RHS is NEVER evaluated.
<code> </code>	Bitwise / Boolean OR	No short-circuit: Evaluates both sides regardless.

```
int a = 10;
if (a < 5 && ++a > 10) { }
System.out.println(a); // Output: 10 (a is NOT incremented because a < 5 was false!)

if (a < 5 & ++a > 10) { }
System.out.println(a); // Output: 11 (Both sides evaluated! a was incremented)
```

9.4 Bitwise Operators & Two's Complement NOT (~)

In two's complement binary representation, the formula for Bitwise NOT is: $\sim x = -(x + 1)$.

```
System.out.println(~5); // Output: -6    (~(5 + 1))
System.out.println(~(-10)); // Output: 9    (~(-10 + 1) = 9)
```

9.5 Bit Shift Operators & The 5-Bit Shift Distance Mask

🔥 ADVANCED INTERVIEW QUESTION: Why is `1 << 32` equal to 1 in Java?

According to JLS §15.19, when shifting a 32-bit `int`, the JVM takes **only the lower 5 bits of the shift distance** (mask: `distance & 0x1F`, i.e., modulo 32).

For a 64-bit `long`, it takes only the lower 6 bits (mask: `distance & 0x3F`, i.e., modulo 64).

```
System.out.println(1 << 32); // Shift distance: 32 & 31 = 0! Result: 1 << 0 = 1
System.out.println(1 << 33); // Shift distance: 33 & 31 = 1! Result: 1 << 1 = 2
```

10. Type Conversion, Widening & Type Promotion

Numeric Widening Hierarchy (Automatic / Implicit):

```
byte  → short
      |
char  → int  → long → float → double
```

Implicit Widening vs Explicit Narrowing

- **Widening (Automatic):** Storing smaller type in a larger type container (No data loss risk).
- **Narrowing (Manual Cast Required):** Larger type into smaller type. Truncates bits/decimals without rounding.

```
double salary = 50000.75;
int truncated = (int) salary; // 50000 (Fraction discarded; truncates towards zero)

double neg = -10.9;
int truncatedNeg = (int) neg; // -10 (NOT -11!)

int overflowVal = 130;
byte b = (byte) overflowVal; // Output: -126 (Wraparound in 8-bit signed space!)
```

The Universal Binary Type Promotion Rules

Whenever binary arithmetic (`+`, `-`, `*`, `/`, `%`) is performed, Java applies automatic numeric promotion:

Rule Priority	Operands Condition	Promoted Result Type	Placement Example
Priority 1	If either operand is <code>double</code>	<code>double</code>	<code>float + double → double</code>
Priority 2	Else if either operand is <code>float</code>	<code>float</code>	<code>long + float → float</code>
Priority 3	Else if either operand is <code>long</code>	<code>long</code>	<code>int + long → long</code>
Priority 4 (DEFAULT)	All smaller integral types (<code>byte</code> , <code>short</code> , <code>char</code>)	<code>int</code>	<code>byte + byte → int</code> <code>char + short → int</code>

🔥 CLASSIC MCQ: Why does `byte c = a + b;` fail to compile?

```
byte a = 10;
byte b = 20;
// byte c = a + b; // ❌ COMPILER ERROR: Cannot convert from int to byte
byte c = (byte)(a + b); // ✅ Explicit cast required!
```

Explanation: Under JLS §5.6, unary numeric promotion always widens `byte`, `short`, and `char` operands to `int` before arithmetic execution.

The Compile-Time Constant Optimization Exception

```
byte b = 10 + 20; // ✓ COMPILES! 10 + 20 is evaluated at compile-time as 30, which fits in byte.  
// byte b2 = 100 + 50; // ✗ COMPILE ERROR: Constant 150 overflows byte range (-128 to 127).
```

Compound Assignment Implicit Cast Trap

```
byte b = 10;  
// b = b + 1; // ✗ COMPILE ERROR (b + 1 produces int)  
b += 1; // ✓ COMPILES! Compound assignment implicitly performs: b = (byte)(b + 1);
```

11. String Concatenation & Expression Precedence

The `+` operator evaluates from **left to right**. If either operand is a `String`, string concatenation is triggered for that sub-expression.

Evaluation Trace for: `System.out.println(10 + 20 + "Java" + 30 + 40);`

Step 1: (10 + 20)	==> 30	(Numeric Addition)
Step 2: (30 + "Java")	==> "30Java"	(String Concatenation)
Step 3: ("30Java" + 30)	==> "30Java30"	(String Concatenation)
Step 4: ("30Java30" + 40)	==> "30Java3040"	(String Concatenation)
Result:	"30Java3040"	

12. Top 8 Placement Coding Problems (Variables & Bitwise)

Problem 1: Swap Two Numbers Without a Third Variable (With Overflow Defense)

Approach A: XOR Manipulation (Optimal for Integers)

```
public static void swapXOR(int a, int b) {
    a = a ^ b;
    b = a ^ b; // b = (a ^ b) ^ b = a
    a = a ^ b; // a = (a ^ b) ^ a = b
    System.out.println("a = " + a + ", b = " + b);
}
```

Placement Note: The arithmetic approach `a = a + b; b = a - b; a = a - b;` can cause integer overflow if `a + b > Integer.MAX_VALUE`. XOR is bitwise and immune to integer overflow.

Problem 2: Check If an Integer Is a Power of Two ($O(1)$ Time)

Powers of 2 in binary have exactly one set bit (e.g., $16 = 10000_2$, $15 = 01111_2$). Subtracting 1 flips all bits after the lowest set bit.

```
public static boolean isPowerOfTwo(int n) {
    return n > 0 && (n & (n - 1)) == 0;
}
```

Problem 3: Count Set Bits (Brian Kernighan's Algorithm — $O(\text{set bits})$)

```
public static int countSetBits(int n) {
    int count = 0;
    while (n != 0) {
        n = n & (n - 1); // Clears the lowest set bit in each iteration
        count++;
    }
    return count;
}
// Note: In production / interviews, also mention: Integer.bitCount(n);
```

Problem 4: Find the Single Non-Repeating Element (All others appear twice)

Leverage XOR properties: $x \oplus x = 0$ and $x \oplus 0 = x$. Time: $O(n)$, Auxiliary Space: $O(1)$.

```
public static int findSingle(int[] nums) {
    int unique = 0;
    for (int num : nums) {
        unique ^= num;
    }
    return unique;
}
```

Problem 5: Reverse an Integer (Guarding Against 32-bit Overflow)

```
public static int reverse(int x) {  
    int rev = 0;  
    while (x != 0) {  
        int pop = x % 10;  
        x /= 10;  
        if (rev > Integer.MAX_VALUE / 10 || (rev == Integer.MAX_VALUE / 10 && pop > 7)) return 0;  
        if (rev < Integer.MIN_VALUE / 10 || (rev == Integer.MIN_VALUE / 10 && pop < -8)) return 0;  
        rev = rev * 10 + pop;  
    }  
    return rev;  
}
```

13. Placement Output Prediction Drills (Rapid-Fire)

#	Code Snippet	Predicted Output	Core Concept / Trap
1	<code>System.out.println(10 / 4);</code>	2	Integer division truncates decimal remainder.
2	<code>System.out.println(10 + 20 + "Hi");</code>	"30Hi"	Addition precedes string concatenation.
3	<code>System.out.println("Hi" + 10 + 20);</code>	"Hi1020"	Left-to-right: Once string appears, all <code>+</code> concatenate.
4	<code>char c = 'A'; System.out.println(c + 1);</code>	66	<code>char</code> undergoes numeric promotion to <code>int</code> (65 + 1).
5	<code>char c = '7'; System.out.println(c - '0');</code>	7	Character ASCII digit conversion trick: 55 - 48 = 7\$.
6	<code>Integer a = 127, b = 127; System.out.println(a == b);</code>	true	IntegerCache hit (-128 to 127).
7	<code>Integer a = 128, b = 128; System.out.println(a == b);</code>	false	IntegerCache miss; two distinct heap instances.
8	<code>int x = Integer.MAX_VALUE; System.out.println(x + 1);</code>	-2147483648	Integer overflow wraps around to <code>Integer.MIN_VALUE</code> .
9	<code>int a = 5; System.out.println(a++ + a++);</code>	11	\$5\$ (a becomes 6) \$+ 6\$ (a becomes 7) \$= 11\$.
10	<code>System.out.println(1 << 32);</code>	1	32-bit shift distance is masked with <code>0x1F</code> (\$32 \bmod 32 = 0\$).

14. 30-Second Quick Revision Cheat Sheet

🚀 Primitive Summary

- **8 Primitives:** byte(1B), short(2B), int(4B), long(8B), float(4B), double(8B), char(2B), boolean(JVM-dep).
- **Default Int Literal:** `int` (append `L` for long).
- **Default Float Literal:** `double` (append `f` for float).
- **Local variables:** NO default values.

🚀 Type Promotion & Wrappers

- `byte + byte → int` (ALWAYS!).
- `char + char → int`.
- **Integer Cache:** -128 to 127.
- **Comparing wrappers:** Always use `.equals()`.
- **Unboxing null:** Throws `NullPointerException`.

✦ Bitwise & Operators

- `~x` equals `-(x + 1)`.
- `x ^ x = 0` and `x ^ 0 = x`.
- Power of 2: `n > 0 && (n & (n - 1)) == 0`.
- `&&` short-circuits; `&` evaluates both sides.
- Shift distance: masked by 31 for int, 63 for long.

✦ Assignments & Memory

- `b += 1` works because of implicit cast `(byte)(b + 1)`.
- `b = b + 1` fails compile check.
- `final Student s` : Reference cannot change, but object fields CAN.
- Stack holds local variables & references; Heap holds all objects.